

```

classDiagram
    class User {
        id: Int
        name: Varchar(100)
        email: Varchar(255) UNIQUE
        password: Char(60)
        privacyCode: Varchar(255) UNIQUE
        isAdminMonitor: Boolean
        createdAt / updatedAt: Timestamp
    }
    class PasswordResetToken {
        id: Int
        userId: Int (FK → User)
        tokenHash: Varchar(64) UNIQUE
        expiresAt: Timestamp
        consumedAt: Timestamp?
        createdAt / updatedAt: Timestamp
    }
    class BigFiveResponse {
        id: Int
        userId: Int (FK → User)
        itemNumber: Int (1-44)
        response: Int (1-5 Likert)
        createdAt: Timestamp
        UNIQUE(userId, itemNumber)
    }
    class BigFiveScore {
        id: Int
        userId: Int (FK → User) UNIQUE
        extraversion/agreeableness: Int
        conscientiousness/neuroticism: Int
        openness: Int
        createdAt / updatedAt: Timestamp
    }
    class Event {
        id: BigInt
        eventType: Varchar(50)
        actorId / teamId: Int?
        entityType: Varchar(50)?
        entityId: Int?
        action: Varchar(50)?
        payload: Json
        schemaVersion: Varchar(10)
        createdAt: Timestamp (immutable)
    }
    User --> PasswordResetToken
    PasswordResetToken --> BigFiveResponse
    BigFiveResponse --> BigFiveScore
    BigFiveScore --> Event

```

```

classDiagram
    class Team {
        id: Int
        name: Varchar(100) UNIQUE
        version: Int (optimistic lock)
        createdAt / updatedAt: Timestamp
    }
    class TeamMember {
        id: Int
        teamId: Int (FK -> Team)
        userId: Int (FK -> User)
        role: Varchar(20)
        joinedAt: Timestamp
        UNIQUE(teamId, userId)
    }
    class TeamInvite {
        id: Int
        teamId: Int (FK -> Team)
        email: Varchar(255)
        status: Varchar(20)
        invitedBy: Int (FK -> User)
        invitedUserId: Int? (FK -> User)
        lastSentAt / errorMessage
        UNIQUE(teamId, email)
    }
    Team "1" -- "N" TeamMember
    TeamMember "1" -- "N" TeamInvite

```

```
graph TD
    Category[Category categories] --> Pillar[Pillar pillars]
    Pillar --> Practice[Practice practices]
    Practice --> PracticePillar[PracticePillar practice_pillars]
    Category --> TeamPractice[TeamPractice team_practices]
    Pillar --> PracticeAssociation[PracticeAssociation practice_associations]
    Practice --> PracticeAssociation
```

**Category (categories)**

- id: Varchar(50)
- name: Varchar(255) UNIQUE
- description: Text
- displayOrder: Int
- createdAt / updatedAt: Timestamp

**Pillar (pillars)**

- id: Int
- name: Varchar(255)
- categoryId: Varchar(50) (FK → Category)
- description: Text
- UNIQUE(name, categoryId)

**Practice (practices)**

- id: Int
- title: Varchar(100)
- goal: Varchar(500)
- categoryId: Varchar(50) (FK → Category)
- method: Varchar(50)
- isGlobal: Boolean
- tags/activities/roles/workProducts: Json
- completionCriteria/metrics/guidelines: Json/Text
- pitfalls/benefits/associatedPractices: Json
- practiceVersion: Int
- imported\*/source\*/rawJson
- UNIQUE(title, categoryId)

**PracticePillar (practice\_pillars)**

- practicId: Int (FK → Practice)
- pillarId: Int (FK → Pillar)
- createdAt: Timestamp

**TeamPractice (team\_practices)**

- id: Int
- teamId: Int (FK → Team)
- practicId: Int (FK → Practice)
- addedAt: Timestamp
- UNIQUE(teamId, practicId)

**PracticeAssociation (practice\_associations)**

- id: Int
- sourcePracticId: Int (FK → Practice)
- targetPracticId: Int (FK → Practice)
- associationType: Varchar(50)
- UNIQUE(source, target, type)

```

classDiagram
    class Issue {
        id: Int
        title: Varchar(255)
        description: Text
        priority: Priority (enum)
        status: IssueStatus (enum)
        teamId: Int (FK → Team)
        createdBy: Int (FK → User)
        decisionText / decisionRecordedBy/At
        evaluationOutcome/Comments
        evaluationRecordedBy/At
        isStandalone: Boolean
        version: Int (optimistic lock)
    }
    class Comment {
        id: Int
        content: Text
        issueId: Int (FK → Issue)
        authorId: Int (FK → User)
        createdAt / updatedAt: Timestamp
    }
    class IssuePractice {
        issueId: Int (FK → Issue)
        practiceId: Int (FK → Practice)
    }
    class IssueTag {
        issueId: Int (FK → Issue)
        tagId: Int (FK → Tag)
    }
    Issue "1" -- "1" Comment
    Issue "1" -- "1" IssuePractice
    Issue "1" -- "1" IssueTag
    IssuePractice "1" -- "1" IssueTag
    
```

```

classDiagram
    class Tag {
        id: Int
        name: Varchar(100) UNIQUE
        description: Text
        type: Varchar(50)
        isGlobal: Boolean
        createdAt / updatedAt: Timestamp
    }
    class TagPersonalityRelation {
        tagId: Int (FK -> Tag)
        trait: Char(1)
        highPole / lowPole: SmallInt
    }
    class TagCandidate {
        problemTagId: Int (FK -> Tag)
        solutionTagId: Int (FK -> Tag)
        justification: Text
    }
    class TagRecommendation {
        tagId: Int (FK -> Tag)
        recommendationText: Text
        implementationExample: Text
    }
    Tag --> TagPersonalityRelation
    TagPersonalityRelation --> TagCandidate
    TagCandidate --> TagRecommendation

```